

Term Break Project

Build-Your-Own Web App: Specification & Requirements Document

Over break, you will design and build a web application of your own choosing using **Next.js** and **Supabase**. There is no restriction on the app's theme or purpose, so build something you're genuinely interested in (a game tracker, a recipe box, a workout log, a movie ranking app, a class directory, anything). What matters is that your project meets the technical requirements below. When we return from break, you will **present your app to the class**: a live demo plus a short walkthrough of how it works.

1. Choose Your Starting Point

Before you start planning, decide which of these two paths you're taking. Both lead to the same technical requirements (Section 4); the only difference is where your data comes from.

Option A: Build Your Own Idea

Come up with your own app concept and design your own Supabase table(s) around it. This is the path to take if you already have an idea you're excited about, for example a game tracker, a recipe box, a workout log, a movie ranking app, or a class directory. You're free to choose any theme, as long as the technical requirements are met.

Option B: Use the Tennis Club Dataset

Don't have an idea, or want to skip designing your own dataset? Anh (Database and API) has put together a ready-made tennis club booking dataset covering courts, members, and bookings, as part of Practice Assessment 2. You can import this data into your own Supabase project and build your Next.js app around it instead, for example a court booking browser, a member lookup tool, or a court availability viewer. The dataset link is in Section 9 (Helpful Resources).

Option	Best if...	Data source
A. Your Own Idea	You already have a concept you want to build	You design and populate your own Supabase table(s)
B. Tennis Club Dataset	You'd rather focus on the build than the data design	Ready-made dataset from Practice Assessment 2 (Section 9)

2. Overview

This document defines the minimum technical requirements your project must satisfy. Think of it as a checklist for the underlying architecture, not a limit on your creativity. You are free to add extra pages, features, styling, and functionality beyond what's listed here.

Item	Detail
Framework	Next.js (App Router)
Database	Supabase (Postgres)
Language	JavaScript or TypeScript
Timeline	Assigned before break, due when class resumes
Deliverable	Working app (hosted or run locally) + live presentation

3. Project Phases

Treat this like a real build, not a single sitting. Work through three phases over break: **Planning**, **Building**, and **Testing**. You don't need to report progress to anyone between now and presentation day, but following this order will save you from painting yourself into a corner the night before.

Phase 1: Planning

Before writing any code, nail down what you're building and what data it needs.

- Pick your app idea and write one sentence describing what it does.
- List the pages your app needs, and mark which ones will be dynamic routes (at least 2).
- Sketch your Supabase table(s): what columns, what type, what's the primary key.
- Decide which dynamic route parameter will be used to fetch which piece of data.
- (Optional but helpful) Rough sketch or wireframe of your main pages, even on paper.

Phase 2: Building

Build incrementally: get one thing working before adding the next.

- Set up the Next.js project and confirm it runs locally.
- Create your Supabase project and table(s); add a few rows of sample/test data.
- Connect Supabase to your app (client setup, environment variables).
- Build your static pages/layout first (navigation, homepage).
- Build your dynamic routes and read their parameters with `useParams` (remember: this needs a Client Component).
- Connect the dynamic route to Supabase so the parameter fetches real data.
- Add interactivity with `useState` and `useEffect` (loading states, forms, filters, etc.).
- Style the app so it's presentable. This doesn't need to be fancy, just clean.

Phase 3: Testing

Before presentation day, deliberately try to break your own app. This is where most last-minute surprises get caught.

- Visit each dynamic route with several different valid parameter values. Does the right data show up every time?
- Try an invalid or missing parameter (e.g. an ID that doesn't exist). Does the app fail gracefully instead of crashing?
- Refresh the page on a dynamic route directly (not just by clicking a link). Does it still load correctly?
- Check loading states: does the page show something reasonable while data is being fetched?
- Click through every link/button at least once to confirm nothing is broken or unfinished.
- Run through your whole presentation once, start to finish, before class.

Phase	Focus	Output
1. Planning	Idea, page list, data model	Clear plan for routes + Supabase schema
2. Building	Implementation	Working app meeting all core requirements
3. Testing	Breaking it on purpose	A stable app ready to demo live

4. Core Technical Requirements

Every project, regardless of topic, must include all of the following. These are the items your teacher will check for during your presentation.

#	Requirement	What it means
1	Next.js App	Project is built with Next.js using the App Router.
2	2+ Dynamic Routes	At least two routes whose URL contains a variable segment (for example, a route built around an ID and another built around a username or slug). Each segment must be a variable, not a fixed page.
3	Supabase Fetching	At least one dynamic route must use its own URL parameter to fetch a matching record from a Supabase table, so different URLs should pull back different data. Only read (GET-style) requests are required for this project; students do not need to insert, update, or delete records, just fetch and navigate existing data.
4	Display the Data	Fetching data isn't enough on its own: the data you get back from Supabase must actually be rendered on the page or component, in a form the user can see (e.g. a title, list, card, or table). Fetching a record and never displaying it does not satisfy this requirement.
5	Client Components	At least one component in the app is a Client Component (marked with "use client" at the top of the file). This matters beyond just being a requirement: <code>useState</code> , <code>useEffect</code> , and <code>useParams</code> only work inside Client Components, so any component using these hooks must be a Client Component.
6	React Hooks	The app must use <code>useState</code> , <code>useEffect</code> , and <code>useParams</code> at least once each, in a way that's meaningful to the app (not just placed to satisfy the checklist). Remember: all three only work inside a Client Component.
7	Presentation	You will demo the working app live and explain your routes, data flow, and at least one design decision you made.

Note on requirement 3: it is not enough to fetch from Supabase somewhere in the app. The fetch must depend on the dynamic route's parameter, meaning different URLs pull back different data from your database.

5. Dynamic Routes: Details

A dynamic route in Next.js is a page whose URL contains a variable segment. Instead of creating a separate file for every possible page, you create one route template and Next.js fills in the variable part from the actual URL the visitor requests.

For example, a single route template for recipes can serve an unlimited number of individual recipe pages, with the specific recipe determined by the ID in the URL. The same idea applies to a second, unrelated route, for instance, one built around a chef's name instead of a recipe ID. Both routes count toward your requirement, whether they live at the top level of your app or are nested a few folders deep.

Each dynamic route needs a way to read the variable part of the URL so your page knows which record to show. This is done with the **`useParams`** hook, which hands back the current value(s) so you can use them, for example, to look up a matching row in your database. Like all hooks, `useParams` only works inside a Client Component, so the component calling it needs "use client" at the top of the file.

6. Fetching from Supabase

Your app needs a Supabase project with at least one table containing the data your app is about (e.g. recipes, workouts, games, books, whatever fits your theme). At least one dynamic route must use its parameter to look up a specific row.

This project only requires reading data (GET-style requests): fetching records and navigating between them through your dynamic routes. You are not required to build forms or features that insert, update, or delete records in Supabase, though you're welcome to add that if you want to go further.

A typical pattern looks like this: when the page loads, it reads the parameter from the URL with **useParams**, then asks Supabase for the row in your table where the matching column equals that parameter. While the request is in flight, the page shows a loading state; once the data comes back, it's stored and displayed. If the parameter changes (the visitor navigates to a different record), the same process runs again automatically for the new value.

This pattern is where the three required hooks work together: **useParams** reads the URL parameter, **useState** holds the fetched data and loading status, and **useEffect** triggers the fetch when the page loads (and re-runs it if the parameter changes). All three of these hooks only work inside a Client Component, so the page or component using them needs "use client" at the top of the file.

Getting the data back from Supabase is only half the job: it also needs to be rendered somewhere on the page, for example as a heading, a list of items, a card, or a table. A fetch that runs successfully but whose result is never shown to the user does not satisfy the requirement.

7. Self-Check Before Presenting

- My app is built with Next.js using the App Router.
- I have at least 2 dynamic routes (routes with variable URL segments).
- At least one dynamic route fetches matching data from Supabase using its own URL parameter.
- The fetched data is actually displayed on the page or component, not just fetched and ignored.
- I have at least one Client Component (marked with "use client") since hooks only work there.
- I use useState somewhere meaningful (e.g. storing fetched data, form input, toggles).
- I use useEffect somewhere meaningful (e.g. triggering a fetch on page load).
- I use useParams to read a dynamic route's parameter and actually use it.
- My app runs without crashing and I can demo it live.
- I can explain, in my own words, how data flows from Supabase to the screen.

8. Presentation Day

When we return from break, each student will give a short live walkthrough of their app to the class.

Your presentation should cover:

- What your app does and why you chose the idea.
- A live demo: navigate between at least two dynamic routes and show real data loading from Supabase.
- A quick look at your Supabase table (what data lives there).
- One challenge you ran into and how you solved it.

Item	Detail
Suggested Time	~4–6 minutes per student (live demo + questions)
Format	Screen share or classroom display, app must actually run
Bring	Laptop with the project ready to run, Supabase project accessible

9. Helpful Resources

- [Next.js Docs \(official\)](#)
- [Next.js Docs: Dynamic Routes](#)
- [Next.js Docs: useParams](#)
- [Supabase Docs: JavaScript Client](#)
- [Supabase Docs: Quickstart](#)
- [React Docs: useState / useEffect](#)
- [Class Course Site: Build Advanced Interfaces](#)
- [Optional Dataset: KangaLife Tennis Club \(Practice Assessment 2\)](#)

Questions before break? Ask in class or reach out to your teacher. Have fun building, pick something you actually want to use.